BA ZC581 :
Applied Data Science
Module: 4

J A Gokhale.

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Disclaimer

- The slides presented here are obtained / extracted from the authors of the prescribed books and from various other subject contributors.
- I hereby thank and acknowledge all the contributors for their material and inputs.
- I have added and modified a few slides to suit the requirements of the course.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Module 4: APIs and File Based Data Collection

Module 4 Topics

Topic 1: Data Collection Methods

Topic 2: JSON, CSV, XML

Topic 3: REST APIs with Python

Topic 4: Scraping External Sources

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 1: Web Data Collection Methods

Methods of Collecting Data from the Web

- Web Scraping
- Application Program Interfaces & Data Feeds
- Transaction Tracking
- Online Surveys
- Online Tracking and Analysis
- Social Media Monitoring
- Online Interviews and Focus Groups
- Repositories and Public Datasets
- Mobile Data
- Secondary Methods

Web Scraping

- Use automated tools or scripts to extract data from websites.
- Web scrapers can be programmed to navigate websites, identify specific elements (e.g., text, images, links), and extract the desired information.
- This method is particularly useful for collecting large volumes of structured or semi-structured data from multiple sources.

APIs for Data Collection

- APIs (Application Programming Interfaces) are fundamental for data collection, offering a structured and programmatic way to access and retrieve data from various sources.
- Instead of web scraping, APIs provide direct access to a service's data, often in formats like JSON or XML, which can be easily parsed for the required content.

Transaction Tracking

- This method involves capture and analysis of data generated during business transactions or interactions.
- It typically focuses on data related to purchase transactions, such as sales, orders, payments, and other transactional activities.
- Transactional tracking relies on automated systems and technologies, such as point-of-sale (POS) systems, online shopping carts, electronic payment systems, and other transactional platforms.
- One of the most high value, accurate and reliable data.

Online Surveys

- Survey Platforms enable the creation and distribution of online surveys to gather data directly from individuals.
  - Platforms - SurveyMonkey, Google Forms, or Typeform
- This method is effective for collecting qualitative and quantitative data, such as opinions, preferences, and feedback from target audiences.

Online Tracking & Analysis

- This method focuses on collecting data about user behavior on websites and applications.
- Tools like Google Analytics, cookies, and web beacons track user interactions, including page views, clicks, time spent on pages, and demographic information.
- This data provides insights into user engagement and website performance.

Social Media Monitoring

- Social media platforms offer a rich source of data for understanding public opinion, trends, and brand perception.
- Social media monitoring tools can track mentions, hashtags, sentiment, and engagement metrics related to specific topics, brands, or individuals.
- Most of the social media sites, like X (formerly Twitter), Facebook, Instagram etc are monitored by companies.

Online Interview & Focus Group

- While traditionally conducted in person, interviews and focus groups can be adapted for online settings using video conferencing platforms (e.g., Zoom, Google Meet).
- This allows for qualitative data collection through direct interaction with participants, regardless of geographical location.
  - Treated a strong opinions
  - Accurate
  - More Authentic

Repositories and Public Datasets

- Many organizations and government agencies maintain online repositories of publicly available datasets.
- These repositories offer a convenient way to access pre-collected data for various research purposes.
- Examples include government data portals, academic research archives, and open data initiatives.
  - Kaggle, UCI ML Repo, data.gov , Indian Open Government Data (data.gov.in)

Mobile Data

- Mobile data collection tools leverage smartphones and tablets to gather data on the go.
- These tools enable users to collect data offline and sync it when an internet connection is available.
- They are ideal for remote areas or fieldwork where traditional data collection methods are impractical, offering features like GPS tagging, photo capture, and form-based inputs.
- Effective for situations where location information is vital.

Secondary Data

- **Secondary data is data that has been used in the past.** The researcher can obtain data from data sources, both internal and external to the organizational data.
- Authenticity of the data cannot be ascertained.

Internal sources of secondary data:
Organization's health and safety records
Mission and vision statements
Financial Statements
Magazines
Sales Report
CRM Software
Executive summaries

External sources of secondary data:
Government reports
Press releases
Business journals
Libraries
Internet

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 2: Data Xchange Formats: JSON, CSV and XML (not necessarily in this order)

Data exchange formats

- Data exchange formats are standardized ways to structure and share data between systems.
- In data collection, they ensure compatibility, readability, and efficient processing across tools and platforms. Makes extraction of relevant data simple and easy. (This is called parsing)

Parsing

- **Parsing** is taking raw data and breaking it down into individual fields for a structured, usable format.
- Example 1: Comma Separated values of an invoice
  - 2025-11-01,Laptop,5,80000

Split values on ,

Date	Product	Units	Price
2025-11-21	Laptop	5	80000

  - Date=2025-11-01, Product=Laptop, Units = 5, Price = 80000
- Example 2: Data Received from sensors in a Plant

2025-11-21/Boiler1/Temperature/350.5/C
2025-11-21/Boiler1/Pressure/15.2/bar
2025-11-21/Boiler2/Temperature/340.0/C

Split values on /

Date	Unit	Parameter	Value	UOM
2025-11-21	Boiler1	Temperature	350.5	C
2025-11-21	Boiler1	Pressure	15.2	Bar
2025-11-21	Boiler2	Temperature	340.0	C

Widely used Formats

Format	Description	Suited for
CSV	Comma-separated values; plain text tabular format	Spreadsheets, sensor logs, time series
XLS	Excel style spreadsheet	Exported data from databases
JSON	Light weight, human readable, self describing, key-value format	APIs, web apps, structured data
XML	eXtensible Markup Language - Hierarchical markup language with recognizable tags	Document Exchange, Config files
YAML	Yet Another Markup Language – human friendly format for structured data	Configs, pipelines, DevOps
Protocol Buffers	Compact Binary schema-based format by Google	High performance APIs; mobile apps
BSON	Binary JSON used in MongoDB	NoSQL databases
Avro, Parquet, MessagePack, HDF5, HTML	Other formats	

CSV Format

- A CSV (comma-separated values) file is a plain text file that stores tabular data
- Each line represents a row and values within a line are separated by commas.
  - The Comma is the most common delimiter, while other characters can also be used (like a /, tab, space , semicolon)
- This format is widely supported by spreadsheet programs, databases, and other applications, making it easy to exchange data between them.
- The first line often contains column headers, and each subsequent line contains the data for one record per line.

Example csv

- The file has a .csv extension
- Can be opened and accessed by any text editor (notepad too)
- Excel also opens the file and ‘formats’ in a tabular manner for convenience.
- All fields need not have values.

people.csv

```
Name,Age,City
Anita,30,New Delhi
Biren,25, Whitefield Bengaluru
Champak,40,"Mumbai, Maharashtra“
Durga,, Chennai
```

If the first row does not contain the header, systems may ‘interpret’ the first data row as the header. Alternately, you may specify that header is not present and you can provide the column names.

Example: csv file access

URL:
https://people.sc.fsu.edu/~jburkardt/data/csv/hw_200.csv

Program:

00A-csv reading from web site and storing in csv.py

XML format

eXtensible Markup Language

- Text based format used to store and transport structured data across systems
- Structure: Data is organized in a tree like structure using custom tags
- Self descriptive, tags describe the data making it readable, searchable, and machine parse-able
- Platform independent, works across various OS, programming languages, applications
- XML parsers are used to extract the data from the file.
  - HTML is a special case of XML which has fixed predefined tags.

Example xml

- The file has a .xml extension
- Can be opened and accessed by any text editor (notepad too) or xml editors. Some browsers also display the file contents.

result.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
    <name>Gokhale</name>
    <course>AI Fundamentals</course>
    <scores>
        <midsem>70</midsem>
        <endsem>81</endsem>
    </scores>
</student>
```

Hierarchy of tags is important for the context of the data. <student> is the root element, while other tags are child elements. Tags are enclosed in a pair of < >. Starting tags have a tag name, and the ending tag has the same tag name preceded by a /
Sometimes </> can also be used as the closing tag

Example xml file

XML file : w3schools.com/xml/simple.xml

w3schools.com/xml/simple.xml

https://www.w3schools.com/xml/simple.xml

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```xml
<breakfast_menu>
  <food>
    <name>Belgian Waffles</name>
    <price>$5.95</price>
    <description>Two of our famous Belgian Waffles with plenty of real maple syrup</description>
    <calories>650</calories>
  </food>
  <food>
    <name>Strawberry Belgian Waffles</name>
    <price>$7.95</price>
    <description>Light Belgian waffles covered with strawberries and whipped cream</description>
    <calories>900</calories>
  </food>
  <food>
    <name>Berry-Berry Belgian Waffles</name>
    <price>$8.95</price>
    <description>Light Belgian waffles covered with an assortment of fresh berries and whipped cream</description>
    <calories>900</calories>
  </food>
  <food>
    <name>French Toast</name>
    <price>$4.50</price>
    <description>Thick slices made from our homemade sourdough bread</description>
    <calories>600</calories>
  </food>
  <food>
    <name>Homestyle Breakfast</name>
    <price>$6.95</price>
    <description>Two eggs, bacon or sausage, toast, and our ever-popular hash browns</description>
    <calories>950</calories>
  </food>
</breakfast_menu>
```

Example: xml file access

"https://www.w3schools.com/xml/simple.xml"

Program:

00B-xml reading from web site and storing in csv.py

JSON format

- JavaScript Object Notation (JSON) is a light- weight text-based format for storing and exchanging data that is human readable and machine parseable.
- It uses key-value pairs and arrays to store and transmit data between a server and web application.
  - **Key-Value Pairs:** Data is organized as name/value pairs, where the key is a string in double quotes and the value can be a string, number, boolean, array, or object.
    - colon : separates a key from its value.
    - A comma , separates key-value pairs within an object or values within an array.
  - **Objects:** A collection of key-value pairs enclosed in curly braces { }.
  - **Arrays:** An ordered collection of values enclosed in square brackets [ ]

JSON Format

Example:

```
{
  "studentDetails": {
    "name": "Joe",
    "age": 16,
    "dept": "computers",
    "hobbies": ["dance", "books", "public speaking", "golf"],
    "isClassLeader": false
  }
}
```

JSON Format

- The file has a .json extension
- Can be opened and accessed by any text editor (notepad too).

result.json

```
{
"student": {
  "name": "Gokhale",
  "course": "AI Fundamentals",
  "scores": {
      "midsem": 70,
       "endsem": 81
    }
  }
}
```

Use double quotes for keys and string values.
No trailing commas after the last item in an object or array.
Use { } for objects and [ ] for arrays.

Example: JSON file access

DEMO

url =
https://jsonplaceholder.typicode.com/users

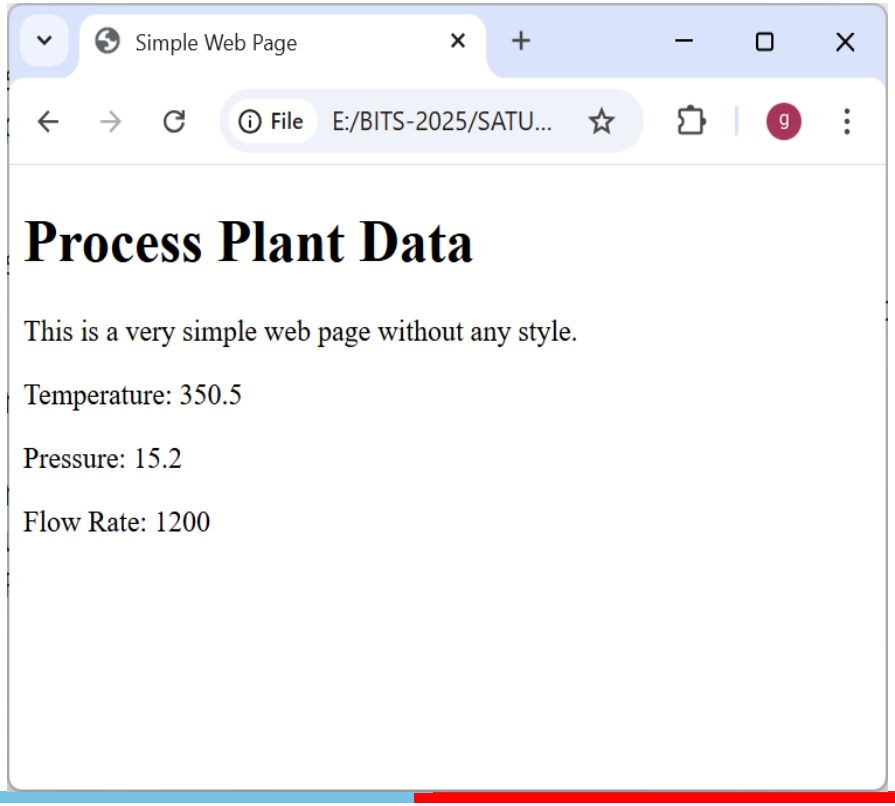
Program:

00C-json reading from web site and storing in csv.py

HTML Format

- Used to define the web page structure and content
- Uses predefined tags to specify how browser should interpret and handle the content.(render web page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Simple Web Page</title>
</head>
<body>
  <h1>Process Plant Data</h1>
  <p>This is a very simple web page without any style.</p>
  <!-- Div grouping numeric values -->
  <div>
    <p>Temperature: 350.5</p>
    <p>Pressure: 15.2</p>
    <p>Flow Rate: 1200</p>
  </div>
</body>
</html>
```



Web page data extraction

- Convert the contents into a hierarchical tree structure
- Data objects are ‘embedded’ within tags
- Lookup for tags and extract value
  - tag_name: value pair is obtained

- Use parsing tools/libraries like BeautifulSoup for python extract these elemental values and store them in csv or xls files.

Differences between JSON, XML and HTML

JSON (JavaScript Object Notation) is commonly used for data storage and transfer. JSON is a popular choice for applications that benefit from a simple and easy-to-use data format.

XML (Extensible Markup Language) is a general-purpose markup language similar to JSON that allows for more complex data structures.

HTML (Hypertext Markup Language) is used to create the structure and content of web pages. You will often see it used with other languages, such as CSS (Cascading Style Sheets) and JavaScript, to unify a website's style and add interactivity to its pages.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

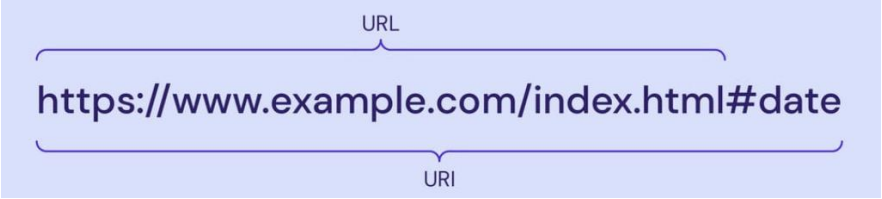
Topic 3: ReST API with Python

The ReST API

- Representational State Transfer Application Program Interface (or REST API or simply API for short).
- Defines a set of protocols that a user can use to query a web service for data.
- Many organizations make their data available through an API.

The ReST API

- The API operates on a client-server model to facilitate communication between different systems over the HTTP protocol.(aka the internet web protocol)
- API interacts with "resources," which are data objects or services (functions to perform a task), identified by unique Uniform Resource Identifiers (URIs).
  - URI (similar to URL) identifies the service running on a remote server:



The ReST API

- The different kinds of information the client can request from the API are called “resources”. Think of a resource as a type of data object returned by the API.
- Endpoint is a specific URL that represents a particular resource or a specific function within the API.
  - It is the address where a client (like a web browser or a mobile app) sends requests to interact with the server and retrieve or manipulate data.
  - Ex: *https://api.example.com/users/1234*

How does ReST API work?

Step 1: Client initiates a request:

A client application (e.g., a web browser, mobile app, or another server) sends an HTTP request to the server. This request specifies:

- **URI:** The address of the resource it wants to interact with (e.g., /users/123 to access user with ID 123).
- **HTTP Method (Verb):** The action to be performed on the resource. Common methods include:
  - GET: Retrieve data.
  - POST: Create new data.
  - PUT: Update existing data.
  - DELETE: Remove data.
- **Headers:** Metadata about the request, such as authentication tokens, content type, or caching instructions.
- **Body (optional):** Data to be sent to the server, especially for POST or PUT requests, typically in formats like JSON or XML.

Example End points

Consider an API for managing users:

- GET https://api.example.com/users This endpoint, using the GET method, would retrieve a list of all users.
- POST https://api.example.com/users This endpoint, using the POST method, would create a new user.
- GET https://api.example.com/users/123 This endpoint, using the GET method, would retrieve the details of a specific user with ID 123.
- PUT https://api.example.com/users/123 This endpoint, using the PUT method, would update the details of the user with ID 123.

How does ReST API work?

Step 2: Server processes the request:

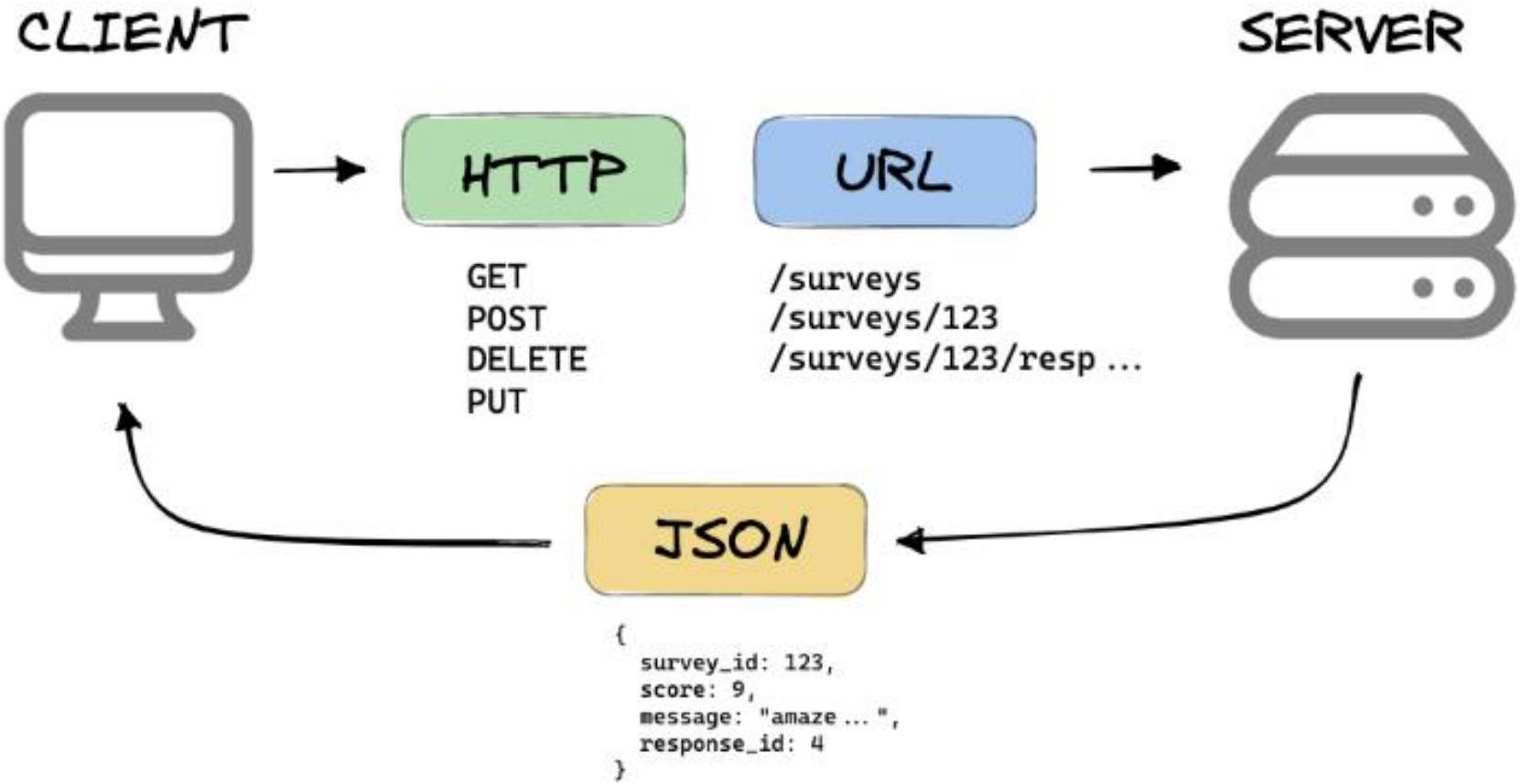
- The server receives the request, authenticates the client, and verifies authorization to perform the requested action.
  - Server gives an error response if it rejects the request due to faled authorization, unavailable resource or failed service execution
- It then processes the request, which may involve
  - querying a database,
  - manipulating data, or
  - executing a specific service that produces a result

How does ReST API work?

Step 3: Server sends a response:

- After processing, the server sends an HTTP response back to the client. This response includes:
  - **HTTP Status Code:** Indicates the outcome of the request (e.g., 200 OK for success, 404 Not Found, 401 Unauthorized).
  - **Headers:** Metadata about the response.
  - **Body (optional):** The requested data or a message, typically in JSON or XML format, representing the "state" of the resource.

How does ReST API Work?



Example

GET https://api.example.com/users/12345?include=posts

Example: Retrieving user data

Request:

```
GET /api/users/12345 HTTP/1.1
Host: api.example.com
Authorization: Bearer your_token_here
Accept: application/json
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "12345",
  "name": "Perry Ostman",
  "email": "p.ostman@example.com",
  "role": "Developer"
}
```

Example API

E-commerce API

```
GET /api/products                    → Browse product catalog
GET /api/products/789                → View specific product
GET /api/products?category=electronics → Filter products
POST /api/cart/items                 → Add item to cart
DELETE /api/cart/items/123           → Remove item from cart
POST /api/orders                     → Create new order
GET /api/orders/456                  → View order details
PUT /api/orders/456/shipping         → Update shipping address
```

Top Benefits of using ReST APIs

HIGH SCALABILITY

PLATFORM INDEPENDENCE

FLEXIBILITY IN DATA FORMATS

IMPROVED PERFORMANCE

UNIFORM INTERFACE

LIGHTWEIGHT

AND MORE

Features of API based data collection

Programmatic Access – application sends requests and receives responses.

Structured Data – data is returned in well defined structure (XML, JSON). Easy to parse

Real time or Historical data – APIs support both these two types of data access

Integration with Other Tools – Data collected can be seamlessly integrated into other applications, or stored in Databases, or application platforms.

Examples of API for web collection

Social Media APIs:

- Twitter API, Facebook Graph API, Instagram API (for specific use cases) enable collection of social media data for sentiment analysis, trend tracking, and audience insights.

Web Analytics APIs:

- Google Analytics API allows programmatic access to website traffic data, user behavior, and other analytics metrics.

Financial APIs:

- Quandl API, Yahoo Finance API provide access to historical and real-time financial data for stocks, commodities, and other financial instruments.

Weather APIs:

- OpenWeatherMap API allows collection of weather data for various locations, including current conditions and forecasts.

E-commerce APIs:

- APIs from platforms like Amazon or eBay allow businesses to collect data on products, pricing, and sales performance.

Specialized Data APIs:

- APIs exist for virtually every domain, from sports statistics (e.g., NBA API) to scientific data and government datasets.

What to consider when using APIs for data collection

API Documentation:

- Thorough understanding of the API's documentation to learn about available endpoints, request parameters, data formats, and authentication requirements is vital.

Rate Limits and Usage Policies:

- Usually there are limitations on the number of requests one can make within a given timeframe to avoid being blocked.

Authentication:

- Many APIs require authentication (e.g., API keys, OAuth tokens) to access data, ensuring secure and authorized access. Paid services charge on plans that take consider the number of requests per day / quantum of data extracted etc.

Error Handling:

- Implement robust error handling in your code to manage potential issues like network errors, invalid requests, or API outages

Demo in Python

Retrieve Stock prices in real time from yfinance

Program 1F

Demo in Python

Get News on a topic

Program 2

Get the weather report

Program 6a

Program 6b

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 4: Scraping External Sources

Web Scraping for External Data

- Web scraping involves using automated tools or scripts to collect underlying HTML data from websites.
- This data is then parsed to extract the relevant information (usually embedded in structures like DIV tag, BODY tag etc).
- This resulting information is structured and stored in CSV files, XLS files, JSON objects or Databases for further analysis.

Web Scraping: How it works?

Fetching: A web scraper is given a URL. It makes a HTTP request to the web server and it fetches the HTML source code content of that page.

Parsing: The scraper analyzes the HTML and uses specific instructions to find and extract the desired data, such as product prices, names, or contact information contained in tables, paragraphs or DIVs.

Outputting: The collected data is then output in a structured format, like a CSV file, a database, or a spreadsheet, making it easy to use for analysis or other applications.

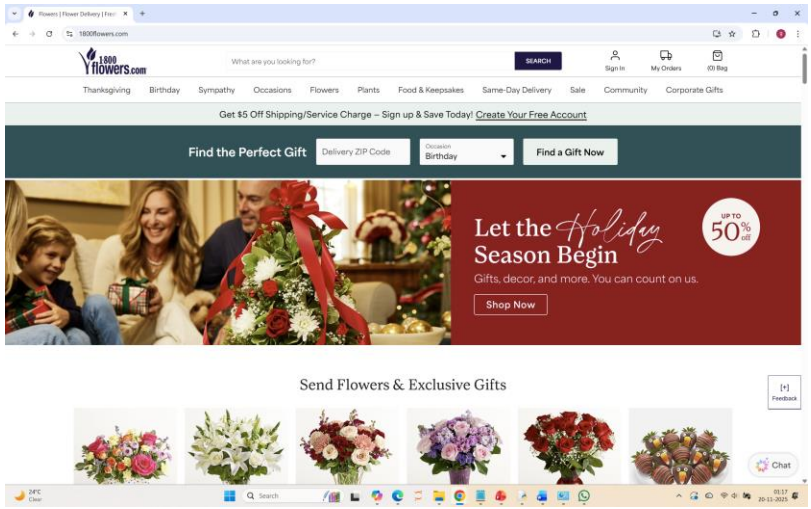
Example: Comparison Site

- Comparison Sites are websites and applications that can help you to easily compare pricing between several retailers for the same product.
- One way that these websites work is by using web scrapers to scrape product data and pricing data from each of the retailers daily.
- Thus, they can provide their users with the comparison data they need.

Web Scraping for External Data

- Popular Libraries for Python
  - Requests
  - BeautifulSoup
  - Scrapy
  - Selenium
- Ethical & Legal Notes
  - Always check a site's robots.txt file
  - Respect rate limits and terms of service
  - Avoid scraping personal or copyrighted data without permission

Example Code (Python)



scraping-test.py - E:/BITS-2025/SATURDAY-APPLIED DATA SCIENCE - 2025/Code Examples/python API calls/scraping-test.py (3.13.1)

File Edit Format Run Options Window Help

```python
# Simple sample code for scraping h2 tags from a web site
import requests
from bs4 import BeautifulSoup

# Target URL
url = "https://1800flowers.com"

# Send HTTP request
response = requests.get(url)

# Parse HTML content
soup = BeautifulSoup(response.text, "html.parser")

# Find all article titles (assuming they are in <h2> tags)
titles = soup.find_all("h2")

# Print each title
for title in titles:
    print(title.text.strip())
```

Ln: 5 Col: 0

```
= RESTART: E:/BITS-2025/SATURDAY-APPLIED DATA SCIENCE -
2025/Code Examples/python API calls/scraping-test.py
This Seasonâs Best Deals
Quick Flower Delivery and Great Customer Service
Order Flowers Online for All Occasions and Holidays
Send Flowers to Someone Internationally
Fresh Flowers Guaranteed by Trusted Florists
Trending Flowers News & Articles from Our Flower Blog,
Petal Talk:
>>>
```

Example

We will scrape details from this web site

https://realpython.github.io/fake-jobs/

Program:

"\python web scraping\webscraping-2-fake-job-site.py"

Summary

- Discussed various types of data exchanges
- Understood the data representation in CSV, XLS and JSON files
- Understood the concept of ReST APIs and their working
- Understood the Studied the working of web scraping

Questions?